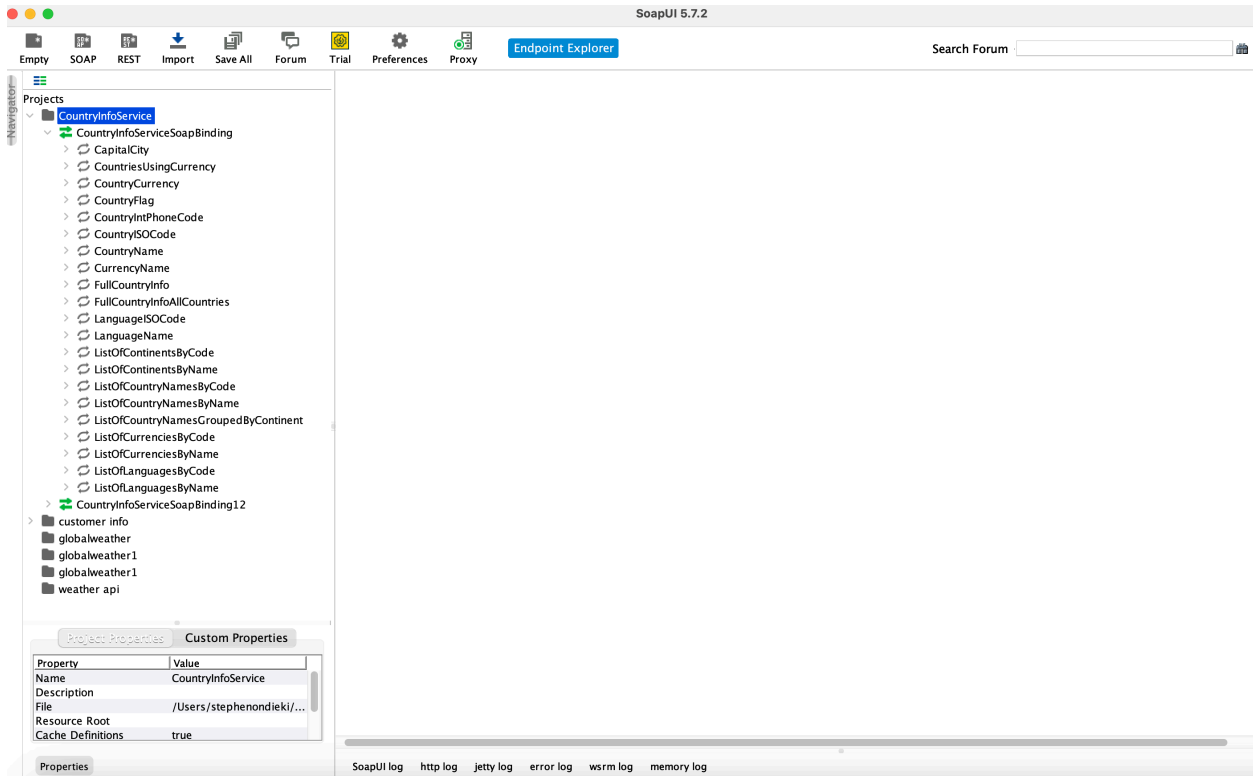




Integration Microservices Engineer

You are required to complete the following tasks within 3 hours. This exercise will test your ability to integrate SOAP and REST APIs, handle data storage in a MySQL database, and implement CRUD operations with error handling using the Spring Boot framework and how to deploy and manage applications on kubernetes cluster

1. Set up a new Spring Boot application:
 - Use Spring Initializr to create a new project if necessary.
 - Include dependencies for Spring Web, Spring Data JPA, and MySQL Driver.
2. Download SoapUI and Import a SOAP Project:
 - Download and install SoapUI.
 - Import the SOAP project using the following WSDL URL: <http://webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso?WSDL>.
 - Once you import the project you should see the screenshot below

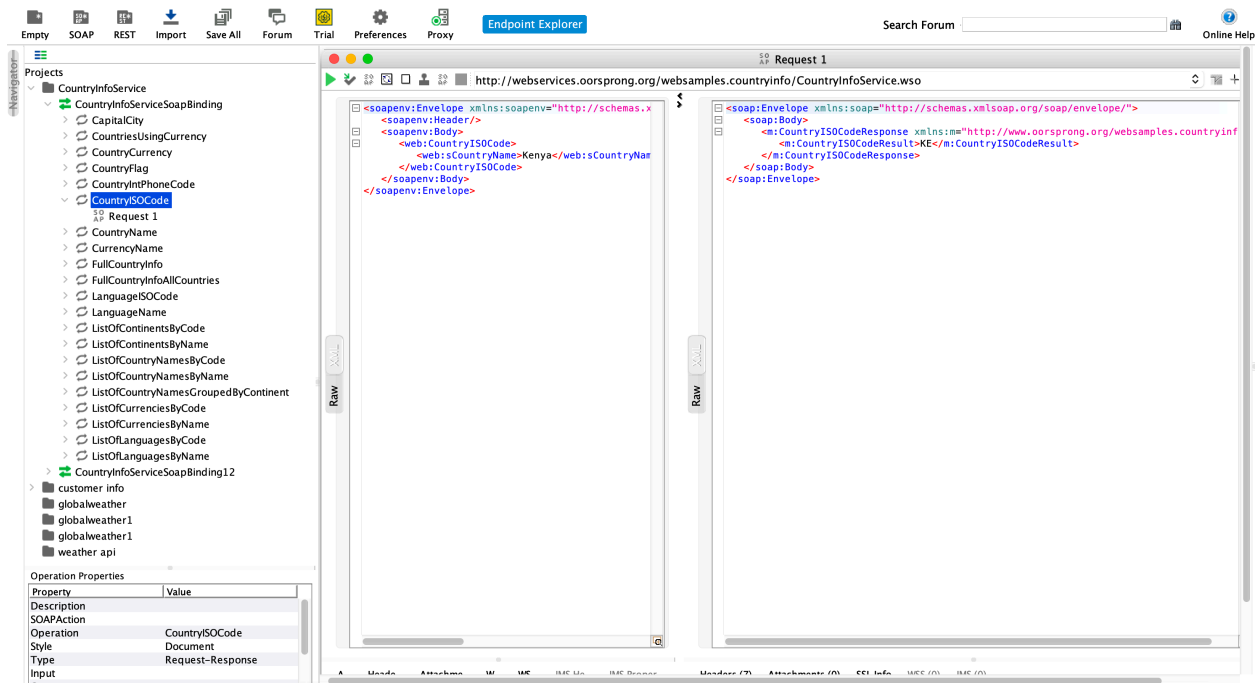


3. Implement a REST controller to receive a country name:

- Create a POST endpoint that receives a JSON object with a name field (e.g., { "name": "Tanzania" }).
- Convert the received country name to sentence case (e.g., "kenya" to "Kenya").

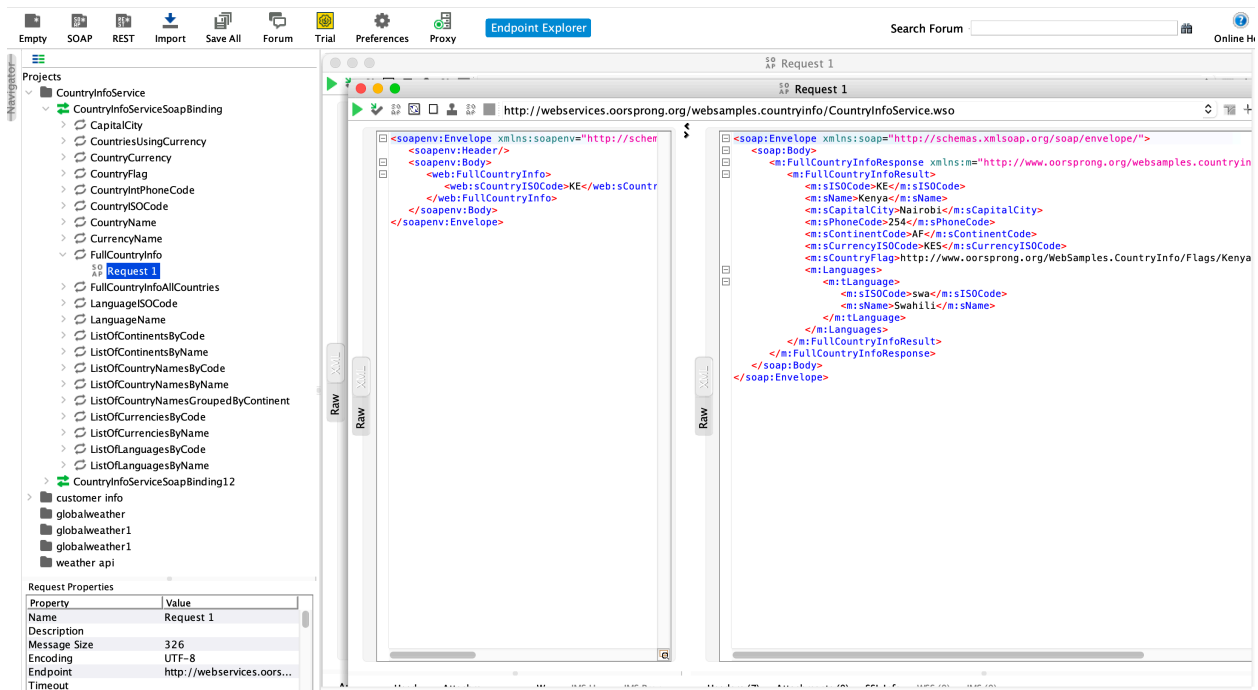
4. Consume the SOAP API to Fetch ISO Code:

- Use the name received in step 3 to call a SOAP endpoint that takes sCountryName as a request body parameter and returns the countryIsoCode as response as shown on the screenshot below:



5. Retrieve and Use the ISO Code to Fetch Full Country Info:

- Extract the countryISOCodeResult from the response in step 4.
- Use the extracted ISO code to call another SOAP endpoint that takes sCountryISOCode as a request body parameter to fetch FullCountryInfo as shown below



6. Create Models to Store the Data:

- Create two models, CountryInfo and Language, to store the data received from step 5.

7. Implement REST APIs for CRUD Operations:

Create REST endpoints to:

- Fetch all country information
- Fetch country information by ID
- Update country information
- Delete country information

8. Create deployment scripts that will be used to deploy the application above on kubernetes.

9. Create a guide on how to deploy the application on kubernetes.

10. Provide a guide on how to troubleshoot the application on kubernetes

Important Notes

- Demonstrate clear system design, including integration patterns, component interactions, and scalability, with justification for key decisions and trade-offs.
- Design for high load using stateless services, horizontal scaling, and appropriate use of caching, queuing, or load balancing.
- Handle failures gracefully using retries, timeouts, circuit breakers, and fallbacks for external service issues.
- Implement structured logging, metrics, and monitoring to enable tracing, debugging, and performance tracking.
- Maintain clear separation of concerns following MVC architecture.
- Include structured, meaningful logs for troubleshooting and visibility.
- Implement robust error handling with proper HTTP status codes and user-friendly responses.
- Ensure production-ready deployment with containerization, configuration management, health checks, and scalability support
- Share the steps of running and testing the application
- Ensure the code is pushed to GitHub by 5pm and link shared